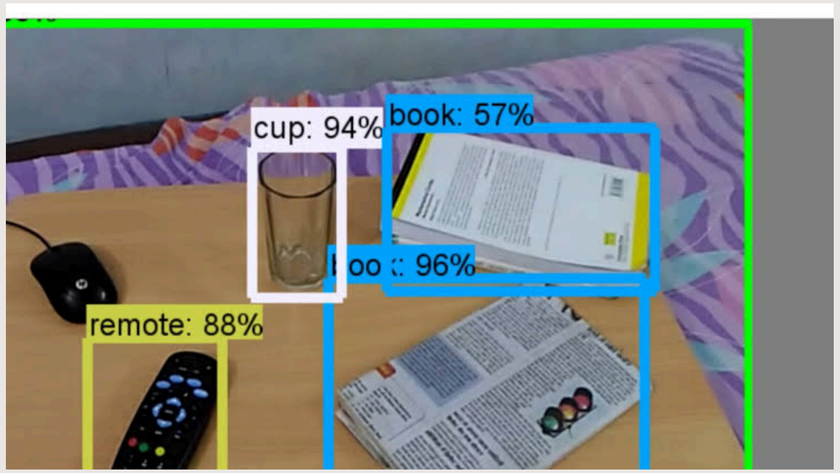
sylvia anter

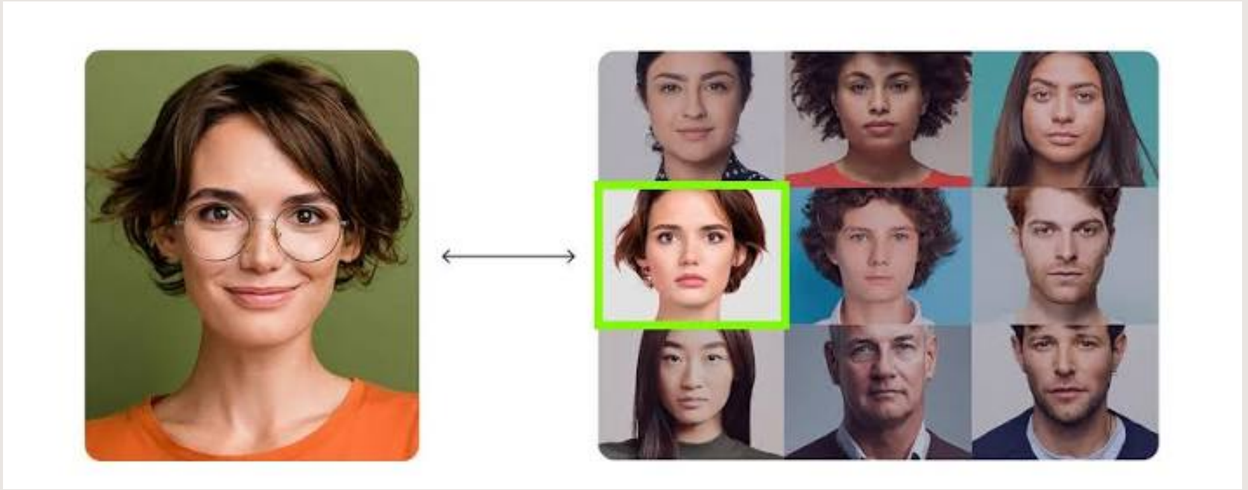
Computer Vision

Computer Vision Applications

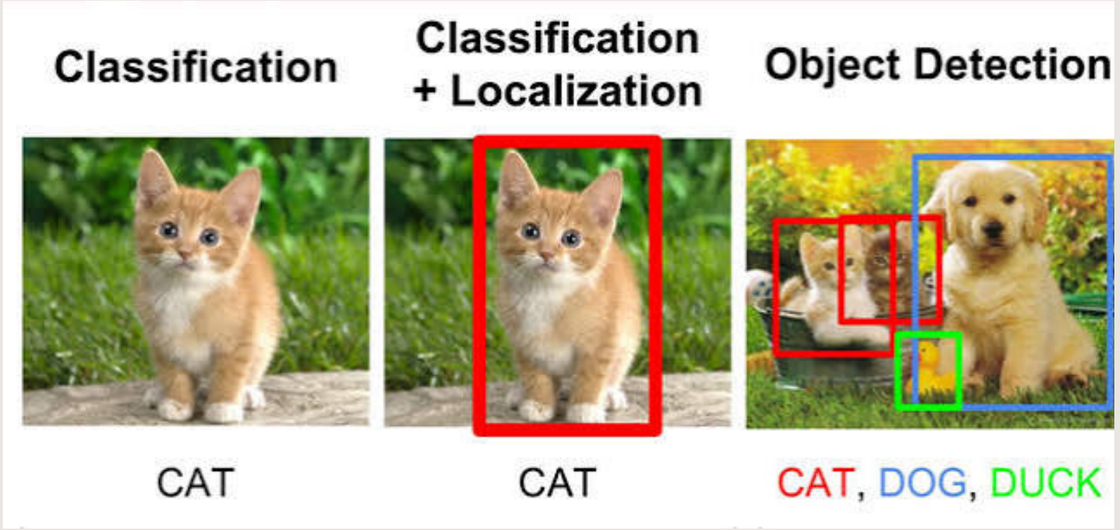
1- object Recognition



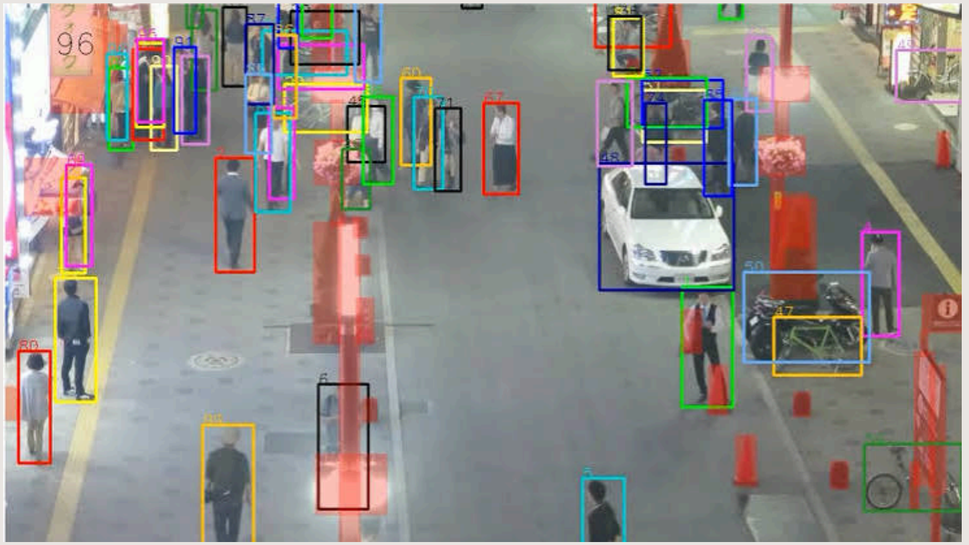
2- face Recognition



3-Image Classification

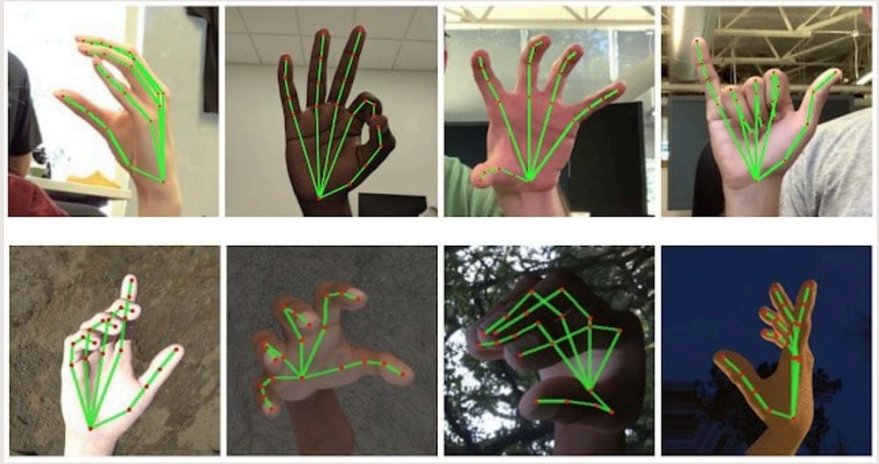


4- object Tacking



Computer Vision Applications

5-gesture Recognition



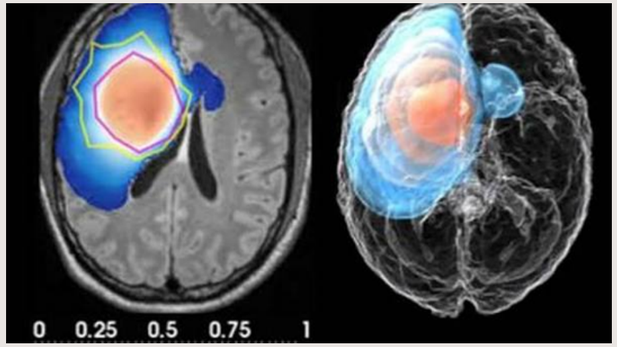
6-scene recognition



7- OCR



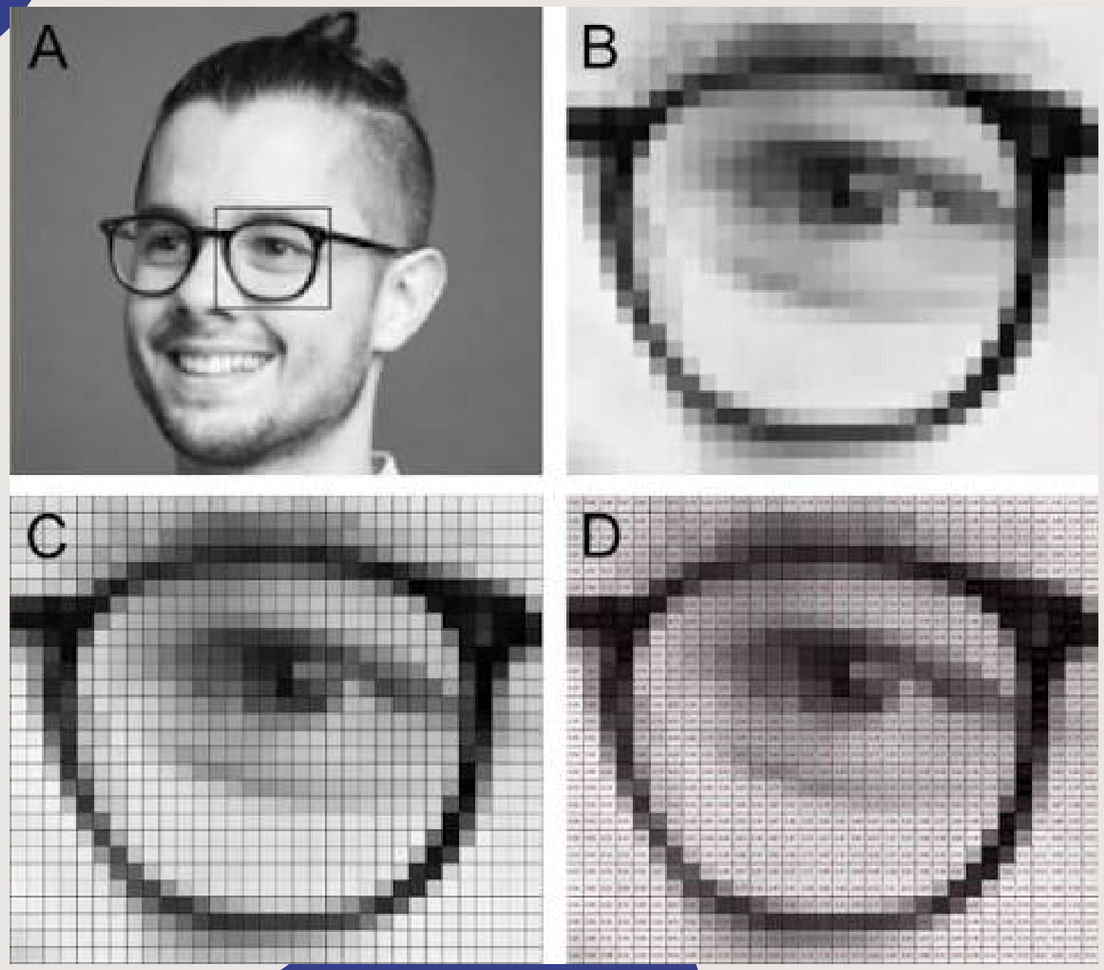
8-Medical Image Analysis

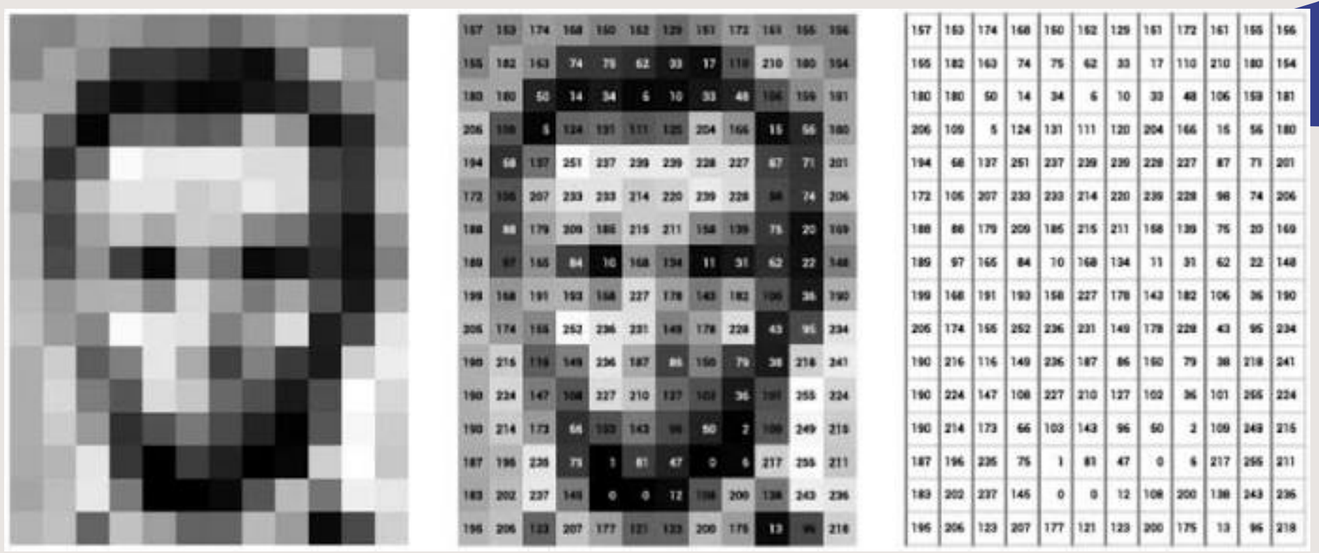


9-Expression Analysis



Open CV





24	24	22	20	18	18	21	21	22	21	19	21	22	22	23
24	25	23	21	19	19	21	21	21	20	18	18	19	21	22
28	29	28	27	24	21	19	20	20	17	14	14	16	17	18
38	37	36	34	30	26	22	20	17	12	9	8	9	10	12
39	39	39	36	32	27	20	14	10	6	4	4	6	6	7
29	31	32	32	31	27	22	18	12	9	5	5	5	5	5
22	22	22	23	23	22	18	16	12	10	7	6	5	3	3
22	20	17	16	14	12	10	8	7	7	8	10	10	8	7
27	25	21	17	12	8	5	3	3	6	10	12	13	12	10

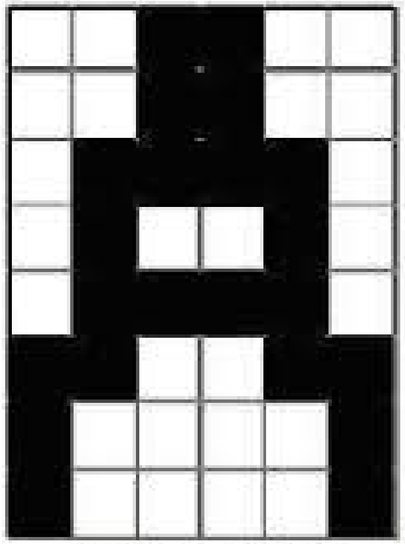
binary

gray scale

color scale



binary



Digitized "A"

0	0	1	1	0	0
0	0	1	1	0	0
0	1	1	1	1	0
0	1	0	0	1	0
0	1	1	1	1	0
1	1	0	0	1	1
1	0	0	0	0	1
1	0	0	0	0	1

Matrix Form of "A"

gray scale



Gray image

Part of the image matrix

52	80	95	103	116	131	141	145	139	138	145
42	67	75	91	100	117	142	148	145	146	148
65	114	103	129	145	145	141	132	143	154	149
81	119	107	141	150	141	141	139	165	155	146
96	117	118	155	147	138	157	151	157	152	142
92	82	140	158	139	142	152	154	162	141	133
116	77	123	136	127	143	156	150	149	139	131
120	78	116	126	122	144	159	148	139	134	132
123	90	126	135	126	143	161	152	138	130	134
144	100	131	137	129	146	168	158	137	130	134
146	102	120	126	130	153	171	156	130	130	131

100*100*3
pixel =
[255,165,0]

color scale

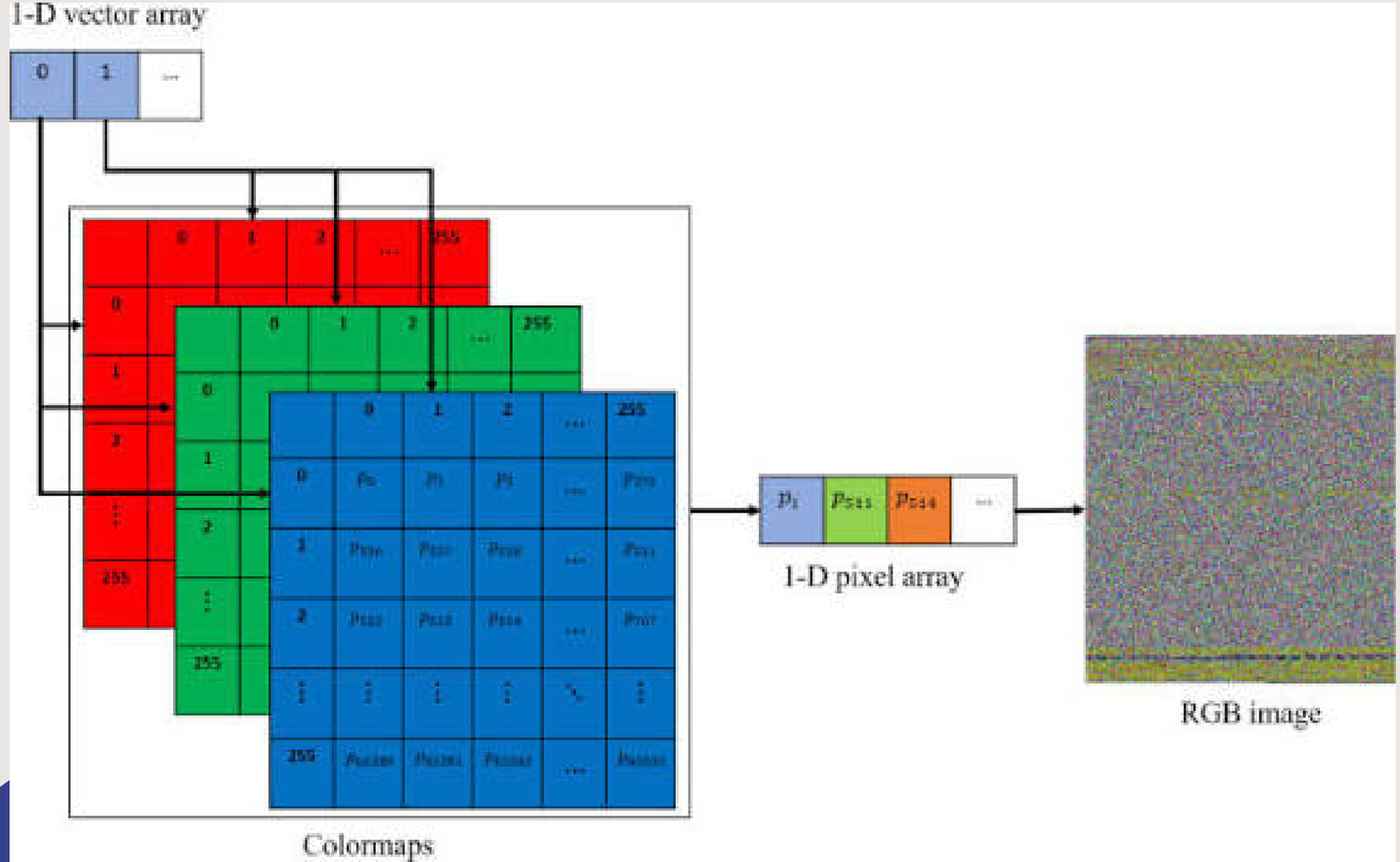
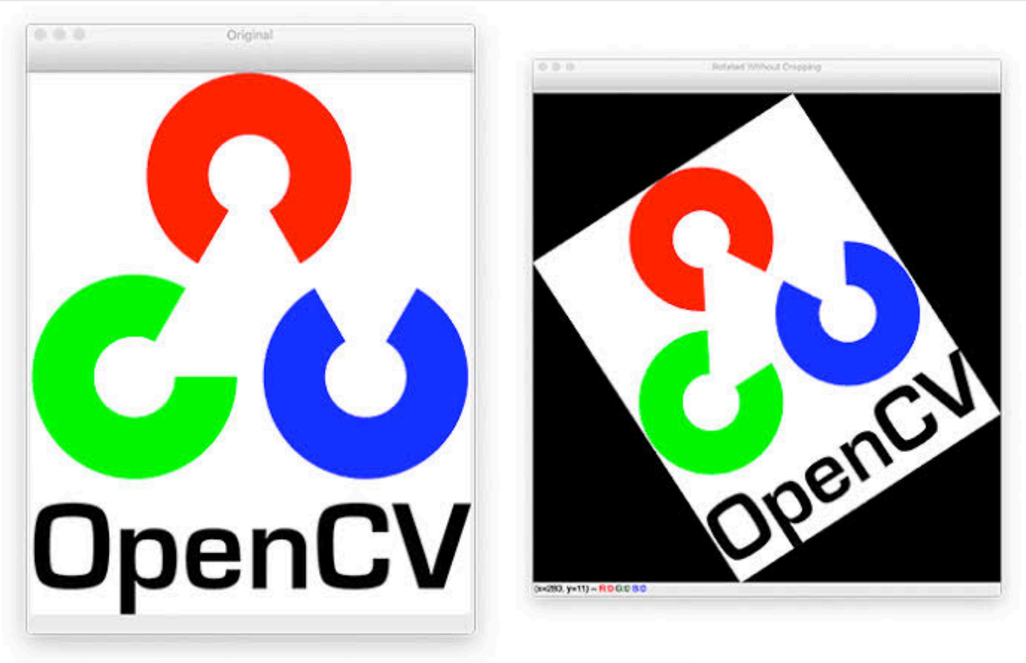


Image Processing

the process of transforming an image into a digital form and performing operations to get some useful information from it

1- Rotation



2- flipping

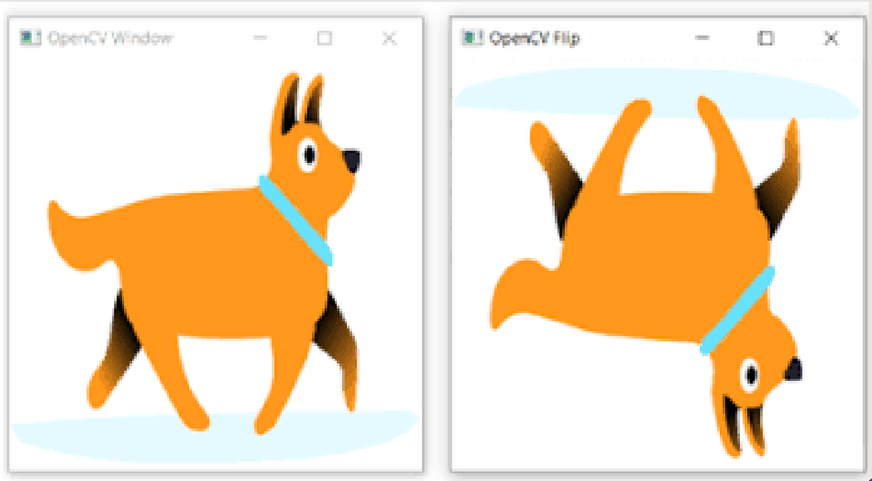
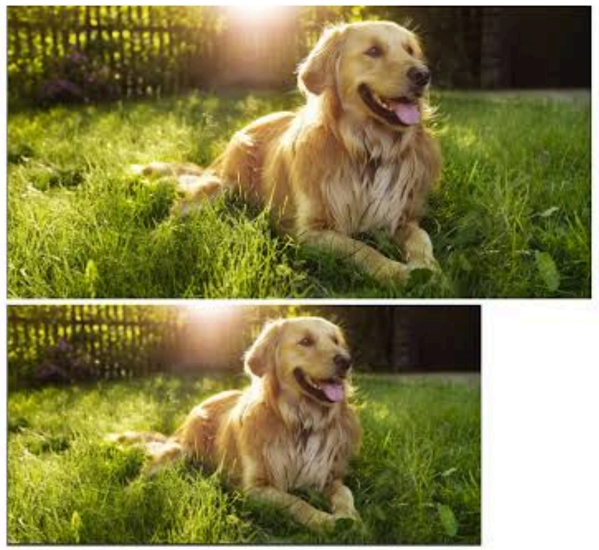


Image Processing

3- scaling



4- cropping

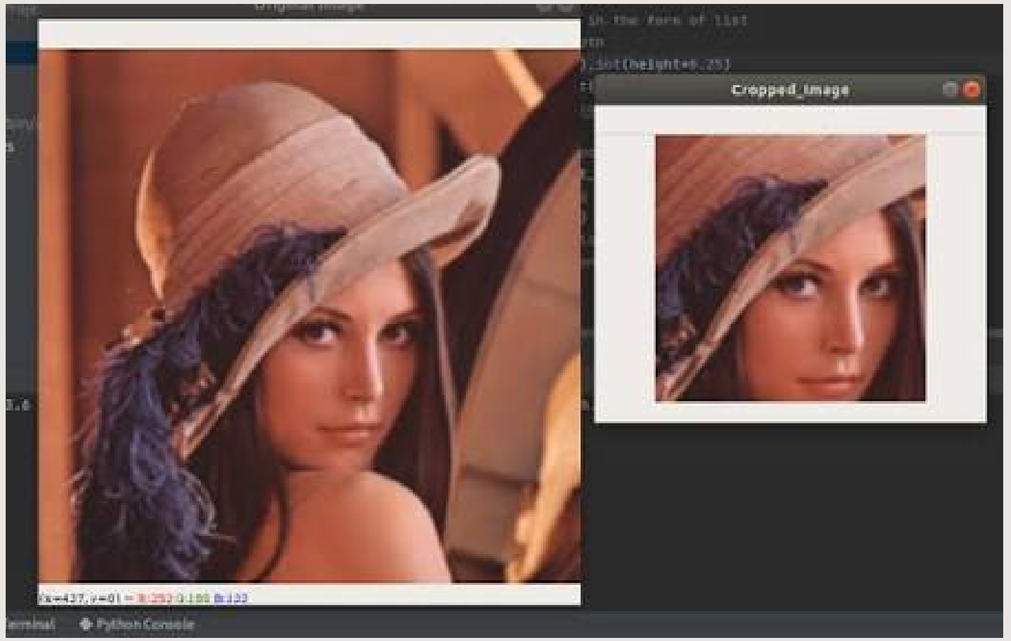
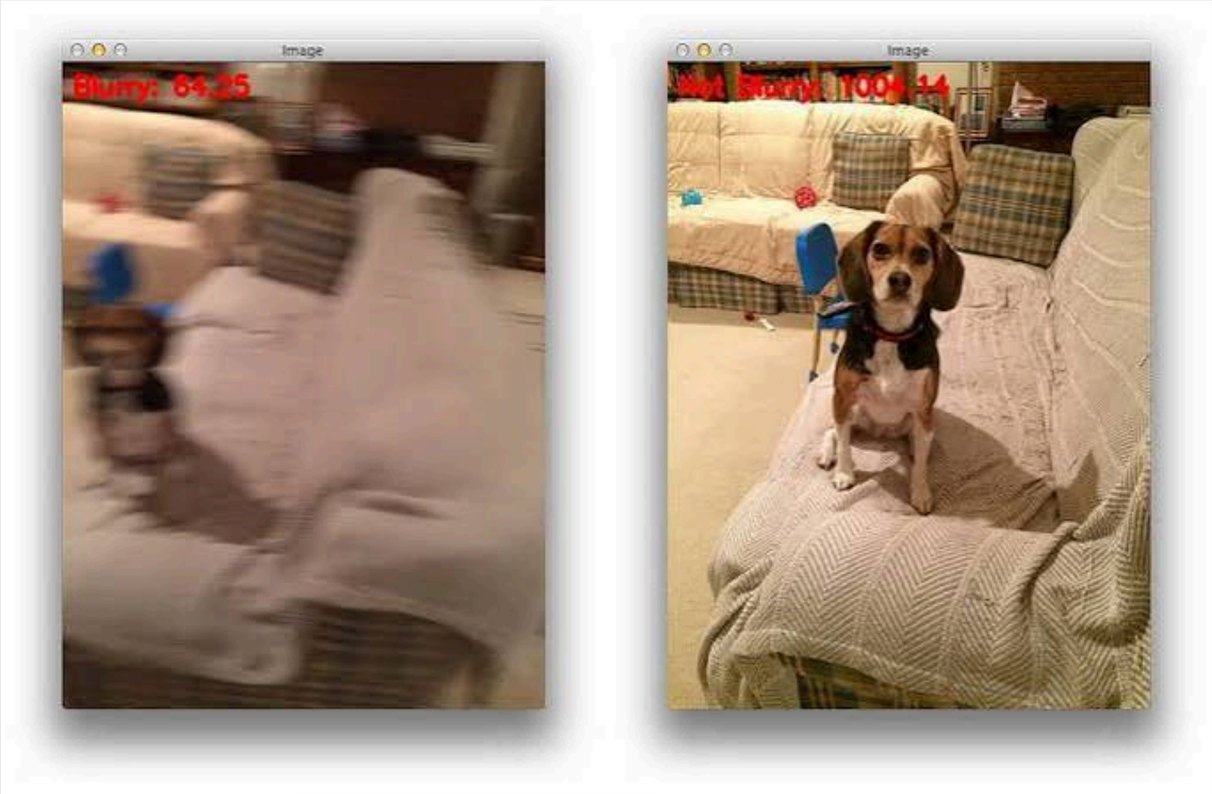


Image Processing

5- blurring



6-sharpening

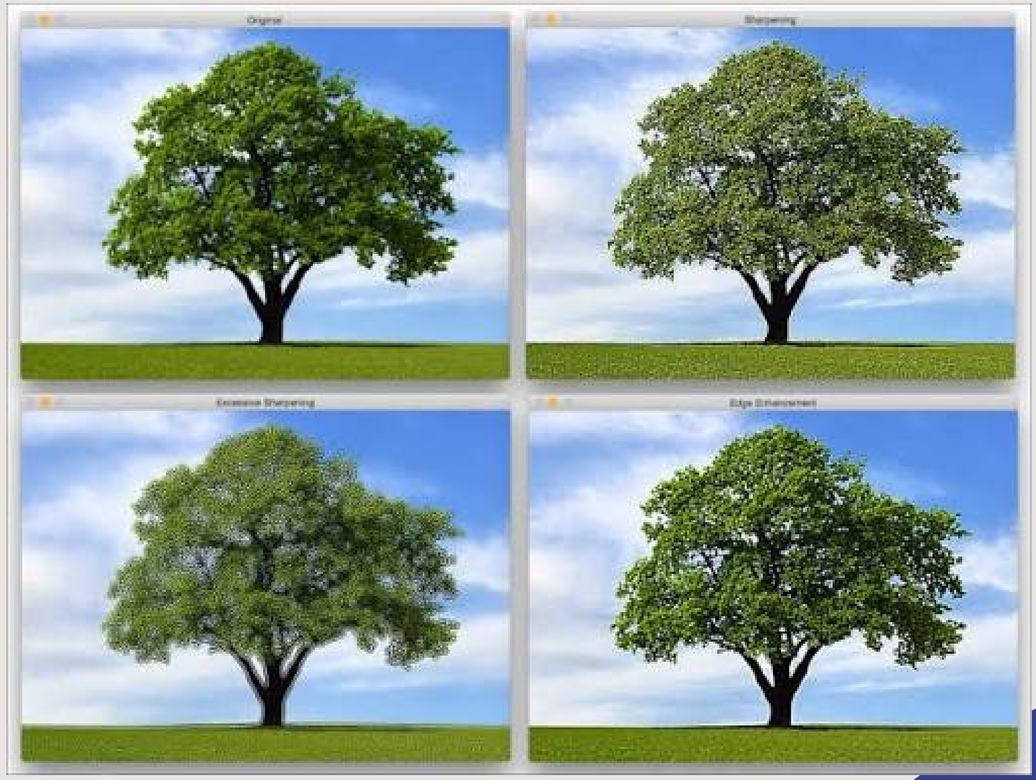
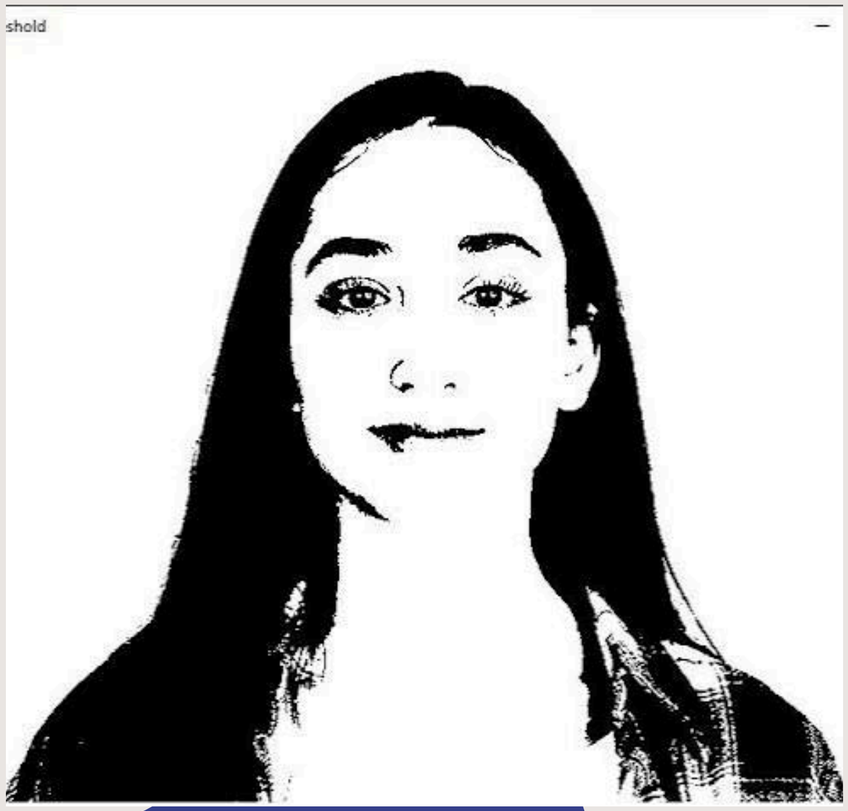
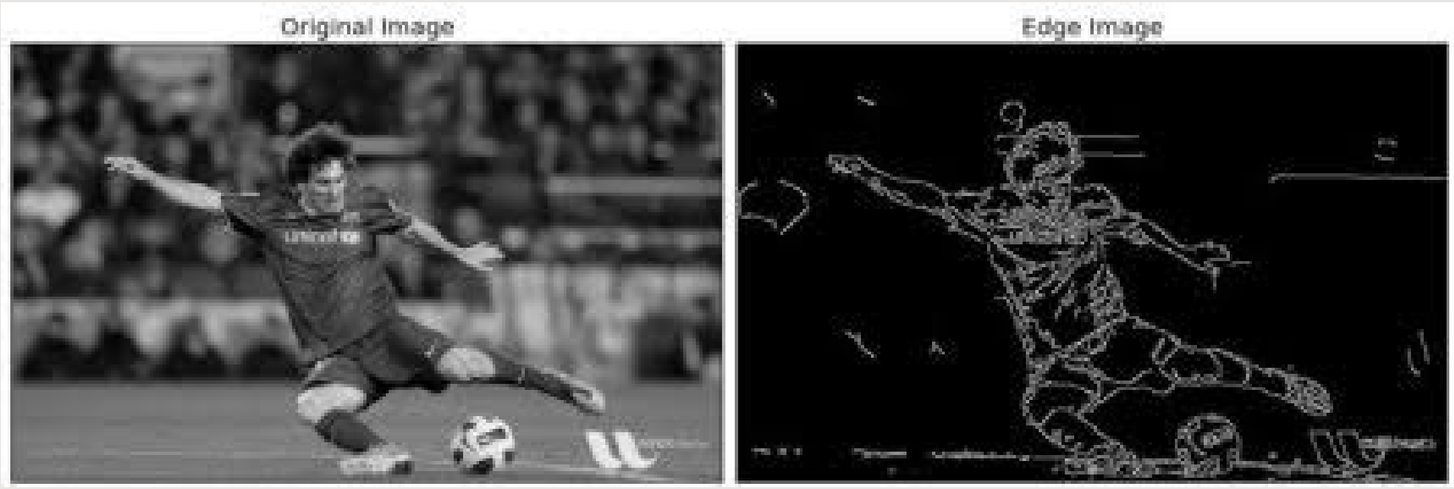


Image Processing

7- threshold (binary image conversion)



8-Canny detection



how to read an image ?




```
image=cv2.imread("/content/dd.jfif")
plt.imshow(cv2.cvtColor(image,cv2.COLOR_BGR2RGB))
plt.show()
```

how to save an image?

imwrite(“ ”)

cv2.imwrite("dd.jpg",image)

Why Convert the image to gray_Scale ?

1- Reduce the Data size.
2- focus on the features not colors.
3- improve preformance.
    1- Canny edge detection 2- Face detection 3- image blurring

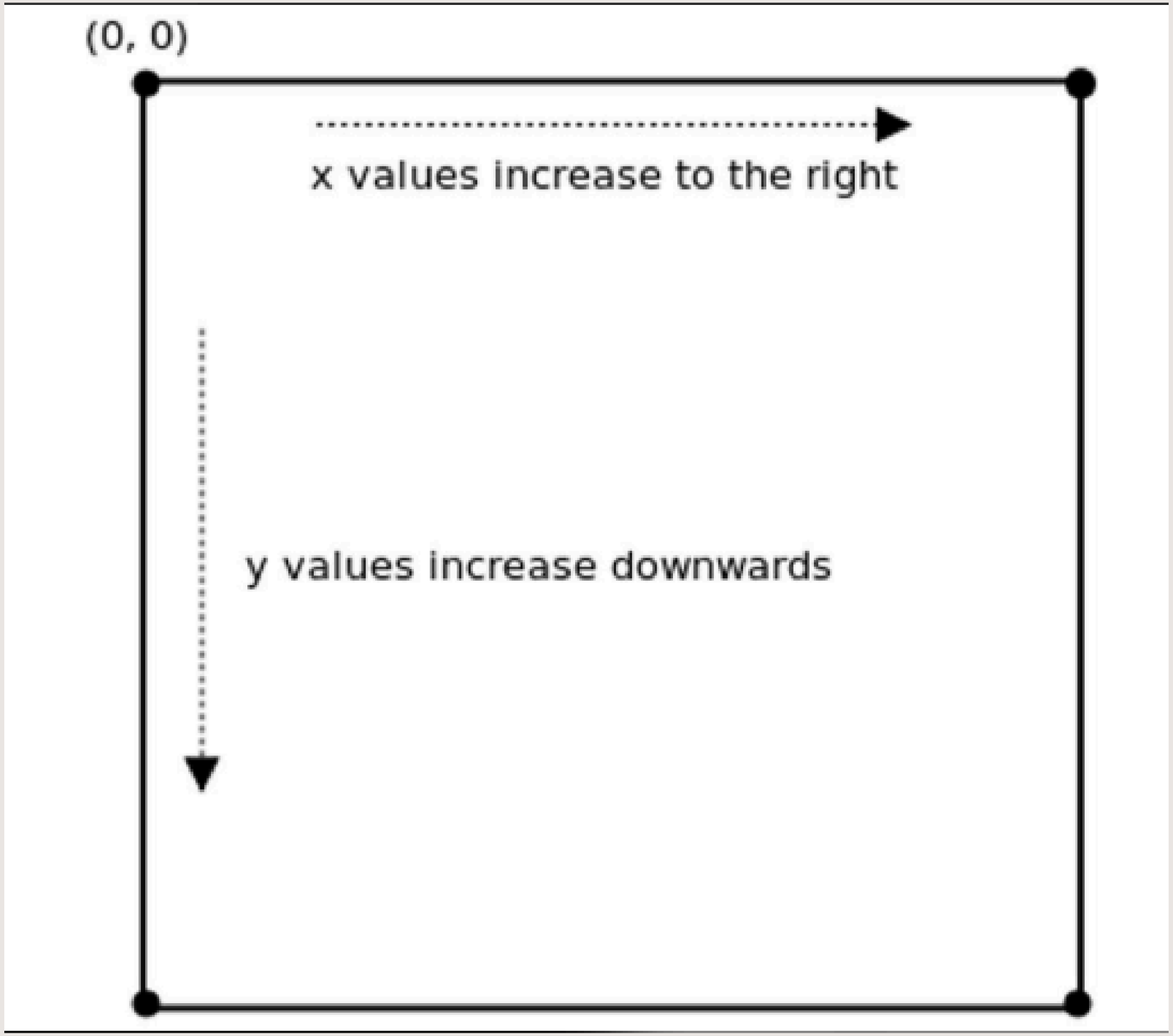
How to apply the gray scale ?

```
img = cv2.imread("image.jpg")

gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

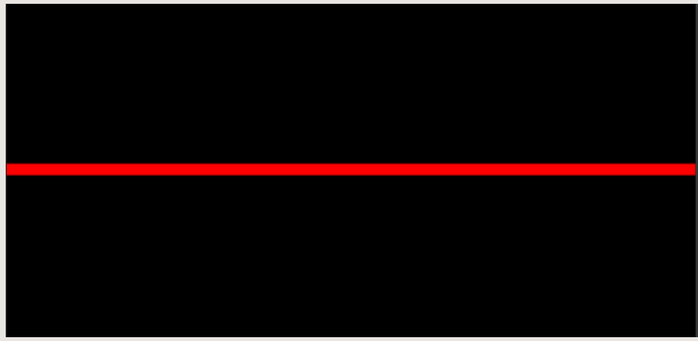
Can We draw on the image using cv2 ?

1- Line
2- Rectangle
3- circle
4- ellipse
5- text

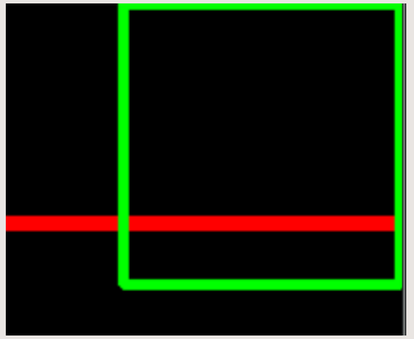

(0, 0)
x values increase to the right
y values increase downwards

Draw a Line

cv2.line(image, start_point, end_point, color, thickness)



cv2.line(img,(0,100),(512,100),(255,0,0),5)



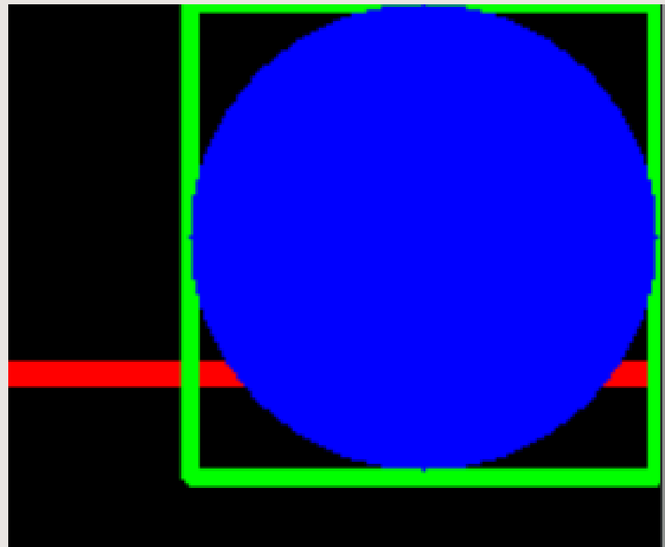
Draw a Rectangle

cv2.rectangle(image, top_left_corner, bottom_right_corner, color, thickness)

cv2.rectangle(img,(384,0),(510,128),(0,255,0),3)

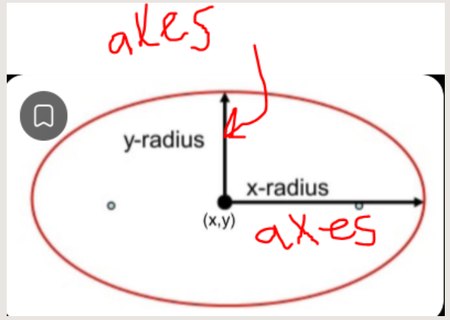
Draw a circle

cv2.circle(image, center, radius, color, thickness)
cv2.circle(img,(447,63), 63, (0,0,255), -1)



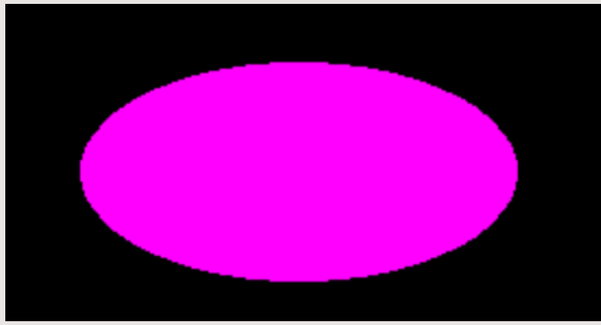
Draw an Ellips

cv2.ellipse(image, center, axes, angle, startAngle, endAngle, color, thickness)



Draw an Ellips

cv2.ellipse(img,(256,256),(100,50),0,0,360,(255,0,255),-1)



Draw an ½ Ellips

cv2.ellipse(img,(256,256),(100,50),0,0,180,(255,0,255),-1)



cv2.ellipse(img,(256,256),(100,50),45,0,360,(255,0,255),-1)

Draw a text

cv2.putText(image, text, org, fontFace, fontScale, color, thickness)

cv2.FONT_HERSHEY_SIMPLEX
cv2.FONT_HERSHEY_PLAIN
cv2.FONT_HERSHEY_DUPLEX
cv2.FONT_HERSHEY_COMPLEX
cv2.FONT_HERSHEY_TRIPLEX
cv2.FONT_HERSHEY_SCRIPT_SIMPLEX
cv2.FONT_HERSHEY_SCRIPT_COMPLEX

Parameter	Value	معناها
image	img	الصورة اللي هنكتب عليها
text	"Hello OpenCV"	النص اللي عايزين نكتبه
org	(50,100)	مكان بداية كتابة النص
fontFace	FONT_HERSHEY_SIMPLEX	نوع الخط (Font Style)
fontScale	1	حجم الخط
color	(255,255,255)	لون النص
thickness	2	سُمك الحروف

Draw a text

cv2.putText(image, text, org, fontFace, fontScale, color, thickness)

font=cv2.FONT_HERSHEY_SIMPLEX

cv2.putText(img,'OpenCV',(10,500),font,4,(255,255,255),2)



Edge Detection

Roberts Operator

بتحسب تغير شدة الاضاء في الاتجاة ال **diagonal**

calculate gradient Diagonal direction

5	6	7	8	9
34	100	67	89	34
64	34	56	76	89
22	12	34	2	34
12	23	56	77	88

+1	0
0	-1

Gx

0	+1
-1	0

Gy

$$I = \begin{bmatrix} 100 & 120 \\ 110 & 200 \end{bmatrix}$$

+1	0
0	-1

Gx

0	+1
-1	0

Gy

G_x = (100 * 1) + (120 *0) + (110 * 0) + (200 * -1)

G_x = -100

G_y = (100 * 0) + (120 *1) + (110 * -1) + (200 * 0)

G_y = 10

$$G = \sqrt{G_x^2 + G_y^2}$$

$$G = \sqrt{(-100)^2 + 10^2}$$

$$G \approx 100.5$$

Edge Detection

Prewitt Operator

calculate gradient in X direction and y direction

بتحسب شدة التغير في الاضاءة في اتجاه معين

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$G = \sqrt{G_x^2 + G_y^2}$$

Edge Detection

$$I = \begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix}$$

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$I \times K_x = \begin{bmatrix} 10(-1) & 10(0) & 100(1) \\ 10(-1) & 10(0) & 100(1) \\ 10(-1) & 10(0) & 100(1) \end{bmatrix}$$

$$\begin{bmatrix} -10 & 0 & 100 \\ -10 & 0 & 100 \\ -10 & 0 & 100 \end{bmatrix}$$

G_x = (−10+100)+(−10+100)+(−10+100) = 270

Edge Detection

Sobel Operator

calculate gradient in X direction and y direction

بتحسب شدة التغير في الاضاءة في اتجاه معين

$$G = \sqrt{G_x^2 + G_y^2}$$

Vertical Edges	Sobel X
Horizontal Edges	Sobel Y

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$


Input Image:
10 9 9 4 0
0 6 6 2 2
5 9 8 4 3
7 5 5 4 3
8 10 8 5 0
Sobel Kernels
-1 0 1
-2 0 2
-1 0 1
X-
1 2 1
0 0 0
-1 -2 -1
Y-

255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Simple Sample Image

$*$

1	2	1
0	0	0
−1	−2	−1

Horizontal Sobel Kernel

$=$

0	0	0	0
1020	1020	1020	1020
1020	1020	1020	1020
0	0	0	0

Horizontal Edges

Sobel operator

Vertical Edges

```
# Sobel X
sobel_x = cv2.Sobel(
    image, cv2.CV_64F, 1, 0, ksize=3
)
```

Horizontal Edges

```
# Sobel Y
sobel_y = cv2.Sobel(
    image, cv2.CV_64F, 0, 1, ksize=3
)
```

Edge Detection

Scharr Operator

Kernel X (Gx)

$$G_x = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

Kernel Y (Gy)

$$G_y = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

$$I = \begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix}$$

G_x = (−30+0+300)
+(−100+0+1000)
+(−30+0+300)

Canny Edge Detection

Noise Reduction | **Gradient Calculation** | **Non-max** | **double thfreshold** | **Edge tracking**

Noise Reduction – Gaussian Blur
sobel gradient calculations
keep only the strong edges remove the rest
edges - strong , weak , non-edge

```
canny = cv2.Canny(img, 100, 200)
display_image('Canny Edge Detection', canny)
```

Image Processing

1- Rotation

$$x' = x\cos(\theta) - y\sin(\theta)$$

$$y' = x\sin(\theta) + y\cos(\theta)$$

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Image Processing

1- Rotation

$$x' = x\cos(\theta) - y\sin(\theta)$$

$$y' = x\sin(\theta) + y\cos(\theta)$$

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Thank You

123-456-7890

www.reallygreatwebsite.com

hello@reallygreatwebsite.com

123 Anywhere., Any City, St 12345